AI Innovate-A-Thon

Topic 10

Aegis

Emergency First-Response Coordination

A campus event exercise prototype

Project Report

Contributor	Registration number
Ayush Pandey	24BIT0394
Dheeraj Sutram	24BIT0416

24 October 2026

Abstract

Aegis is an incident coordination application for synthetic campus event exercises. It organises incoming text reports into provisional incident groups, displays the source evidence, suggests a priority using explicit rules, and records the coordinator's decisions. Reports are stored before analysis so incomplete inputs and model failures remain visible. The coordinator can confirm or reject a grouping, move reports, merge or split incidents, change priority, and review conflicting accounts.

The application uses Python, FastAPI and SQLite for analysis, API access and persistence, with a React and TypeScript interface. An optional local MiniLM encoder supports semantic matching. The measured build uses a deterministic lexical-hash fallback with location, time and category checks. On a fixed synthetic test set of 60 reports across 20 incident families, contextual lexical grouping achieved pairwise precision of 1.000, recall of 0.621 and F1 of 0.766. It produced zero false merge pairs and 25 missed pairs. All reports carried review flags in this mode. The implementation therefore supports inspectable grouping and correction while retaining substantial human review. Evaluation is limited to synthetic fixtures and does not establish suitability for live emergency operations.

Keywords: incident coordination, report grouping, human review, evidence retention, synthetic evaluation.

Contents

1 Introduction **1**

2 Requirements and system design **1**

3 Report analysis and grouping **3**

4 Priority, conflict and coordinator review **4**

5 User interface and demonstration **5**

6 Evaluation **6**

7 Implementation verification **8**

8 Limitations and further work **8**

9 Conclusion **9**

1 Introduction

1.1 Problem and intended user

During a campus event, organisers may receive several accounts of the same situation. One person may identify a door, another may describe a queue, and a later report may say that access has been restored. A report stream leaves the coordinator to reconcile wording, location, timing and uncertainty while keeping track of separate incidents.

Aegis gives one coordinator a shared board for this work. Each incident contains the original reports, proposed membership, supporting comparisons, priority reasons and review history. The interface also shows reports with unknown locations and incomplete descriptions. The bundled map is a schematic of an illustrative event site.

1.2 Objectives

The project implements Topic 10 of the AI Innovate-A-Thon brief. Its objectives are to:

1. preserve every accepted report and its original text;
2. propose incident groups using text and event context;
3. expose the reasons behind grouping and priority suggestions;
4. retain conflicting and changing accounts for inspection;
5. support coordinator corrections with a persistent audit history; and
6. evaluate grouping and review behaviour on reproducible synthetic fixtures.

The prototype accepts English text through a manual form and a controlled replay. All bundled people, reports and incident labels are synthetic. The application has no emergency service connection or responder dispatch function.

1.3 Project contribution

The contribution is the implemented coordination workflow: report preservation, contextual grouping, visible uncertainty, conflict inspection and reversible human decisions in one application. Sentence embeddings and cosine similarity are established methods. Sentence-BERT provides a basis for comparing sentence representations [1], and the optional Aegis adapter uses an existing pretrained model. The project does not train a new encoder.

2 Requirements and system design

2.1 Functional requirements

Table 1 relates the central requirements to their implemented behaviour. The system distinguishes an accepted report, a proposed incident membership and a coordinator decision. This distinction lets the board retain uncertainty without discarding the underlying account.

Table 1: Core requirements and implementation

Requirement	Implemented behaviour
Report intake	Validated text, optional zone, sublocation and event time, with an idempotency key.
Incident grouping	Incremental comparisons, context constraints, an anchor check and a candidate margin.
Visible evidence	Original text, component scores, missing fields and decision reasons.
Explicit priority	Versioned exercise rules with supporting source phrases.
Human correction	Confirm, reject, move, merge, split, priority override and status change.
History	Transactional updates with reasons, before/after values and event ownership.
Reproducible exercise	Step-controlled replay through the ordinary ingestion path.

2.2 Architecture

The Python backend contains the API, analysis engine, SQLite storage and evaluation pipeline. The frontend uses TanStack Start, React and TypeScript. Vite and Bun provide frontend development and build tooling. The interface uses shadcn components, Base UI primitives and self-hosted Figtree fonts.

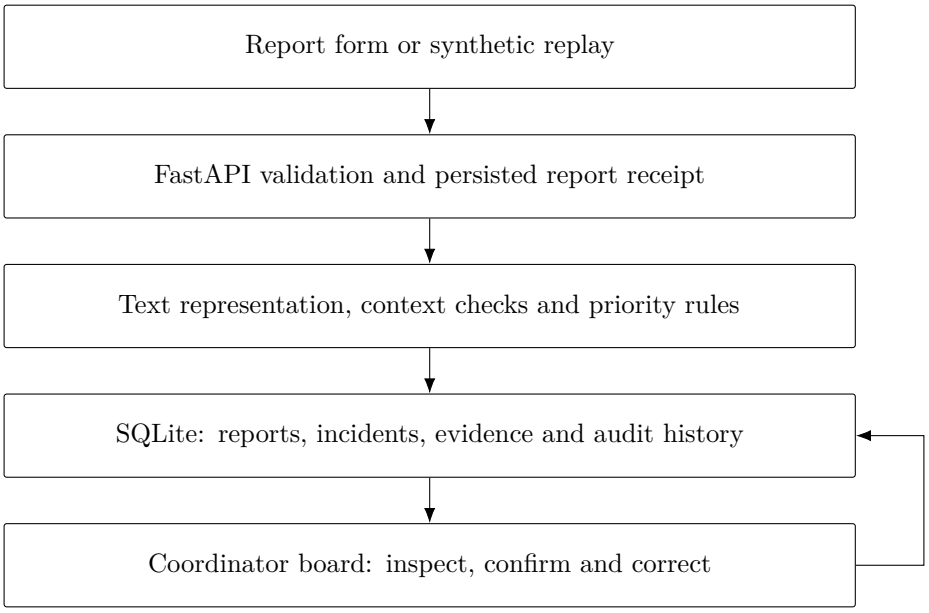


Figure 1: Aegis processing and review flow. Coordinator changes return through the API and storage transaction.

The frontend polls a snapshot endpoint and applies successful mutation responses. The backend exposes the active matching mode through its health endpoint. The application binds to the local host by default and uses a single SQLite database for the exercise state.

2.3 Persistence and invariants

An accepted report is committed with a provisional incident before analysis. Its original text is retained exactly. Analysis adds category, claims, a representation and review reasons. If analysis fails, the report remains accessible with a visible failure or review state.

The storage model separates events, reports, report analysis, incidents, memberships, matching evidence, conflicts, review decisions, rejection constraints, audit events and replay sessions. Each report has one current membership. Historical evidence keeps its original comparison while remaining accessible through the report's current group.

Idempotency is scoped to the event. Repeating a key with the same payload returns the existing receipt. Reusing it for a different payload returns a conflict. Coordinator mutations validate incident versions and commit state changes together with their audit entries. A new replay creates another event and preserves earlier history.

3 Report analysis and grouping

3.1 Text representations

The optional semantic adapter loads `all-MiniLM-L6-v2` from a local model cache. The model maps text to 384-dimensional vectors [2]. Sentence Transformers supports comparison of these representations using cosine similarity [3]. Aegis normalises vectors and records the representation revision with each analysis.

When the optional package or weights are unavailable, or inference fails, the engine uses a deterministic 512-dimensional lexical-hash representation. It hashes token counts into a fixed vector and compares the resulting vectors. This mode relies on lexical overlap and can miss differently worded accounts. The dashboard and evaluation identify the fallback explicitly. MiniLM weights were unavailable in the measured environment, so the results in Section 6 describe lexical grouping.

3.2 Context and candidate checks

Candidate comparisons use the same exercise event. Location checks distinguish a broad zone from a specific shared sublocation or described entity. Time checks use compatible event timestamps, with a default window of 20 minutes and a 120-minute window for missing-person cases. Category and entity checks identify incompatible accounts. Unknown fields are recorded as missing.

Automatic attachment requires known compatible time and specific shared context. A conflict in location, time, category or entity can block attachment. The incoming report is checked against group members for incompatibilities. The first member acts as the anchor, and its text similarity must pass the active mode's minimum. This prevents a chain of individually similar reports from bypassing the anchor requirement.

3.3 Ranking score

For the available components, the pair score is

$$S(r_i, r_j) = \frac{\sum_{k \in A} w_k s_k(r_i, r_j)}{\sum_{k \in A} w_k}, \tag{1}$$

where A contains the available text, location, time and category components. Their weights are 0.65, 0.20, 0.10 and 0.05 respectively. With complete inputs, the denominator is one. Missing components are omitted for ranking, while required context gates still apply. The score is an uncalibrated ranking value.

An incident's candidate score is the mean of comparisons with up to its first four members. Table 2 gives the implemented attachment thresholds. The best eligible candidate must also

exceed the runner-up by at least 0.08. Without a qualifying candidate, the incoming report remains in a separate reviewable group and may retain suggested matches.

Table 2: Attachment thresholds in the current matching configuration

Gate	MiniLM mode	Lexical mode
Minimum candidate score	0.82	0.33
Minimum anchor text similarity	0.62	0.05
Minimum candidate margin	0.08	0.08

The lexical thresholds were developed using the development fixtures. Version 2 adds the 0.05 anchor requirement to prevent shared context alone from attaching textually unrelated reports. This choice makes the fallback conservative and leaves more same-family pairs for manual review.

3.4 Decision procedure

The ingestion and matching procedure is:

1. Validate the payload and event-scoped idempotency key.
2. Commit the original report and its provisional incident.
3. Extract supported claims, assign the exercise priority and create the text representation.
4. Compare eligible incident members and store the component evidence.
5. Reject incompatible candidates and coordinator-rejected pairings.
6. Apply the anchor, context, candidate score and margin gates.
7. Save the proposed membership, priority and review flags. Preserve a separate incident when the gates fail.

Grouping remains provisional until the coordinator reviews it. A coordinator rejection adds a constraint so a later matching pass cannot silently recreate that pairing. Late analysis also preserves decisions made while the report was being processed.

4 Priority, conflict and coordinator review

4.1 Exercise priority policy

Priority is computed using explicit text rules and retained source phrases. Table 3 summarises the implemented labels. These labels organise a synthetic review queue and require human interpretation.

Table 3: Exercise priority labels

Label	Basis
P0	A supported affirmative phrase alleging immediate danger.
P1	A blocked exit, a medical concern, or a missing-person report with a supported vulnerability cue.
P2	A facility issue with a reporter-supplied zone.
Unassessed	Insufficient information or no supported affirmative policy rule.

Rules account for supported negations and hypothetical language. Equipment terms such as “fire extinguisher” and “smoke machine” are excluded from the corresponding standalone danger

trigger. These checks cover a finite English vocabulary. The incident assessment retains the strongest unresolved priority signal across its reports.

4.2 Conflicting and changing accounts

Claim extraction retains a field, value and original supporting phrase. Opposing values can produce a conflict entry with the relevant report identifiers and source spans. An exit described as blocked and later described as clear remains inspectable as a changing or conflicting account. The system cannot establish which account is true from report volume alone.

A later reassuring report does not automatically close the incident or cancel unresolved urgent evidence. The coordinator can resolve or reopen the conflict with a reason. Conflict counts refer to opposing report pairs, so several pairs can arise within one incident.

4.3 Coordinator actions and audit

The coordinator can confirm selected membership, reject a pairing, move reports, merge incidents or split selected reports into a new group. Priority, workflow status and conflict review have separate actions. Each mutation requires a reason and the version of every affected incident that the coordinator reviewed.

A stale request receives an HTTP 409 response. The coordinator must inspect the changed state before rebasing the action. Grouping changes update memberships, evidence access, conflict state and rejection constraints within a transaction. Audit records contain the actor, action, affected identifiers, reason, before/after values and timestamp. This is an application audit history with ordinary database administration limits.

5 User interface and demonstration

The Operations screen combines incident counts, the queue, an illustrative map and the selected incident inspector. Priority labels accompany colour. Unknown-location reports remain in a visible tray, and keyboard users can select map markers. The New report page accepts fresh synthetic wording with optional location and event time.

The replay advances one fixture at a time through the ordinary intake API. It does not pass expected incident labels to the matcher. The demo sequence includes duplicate accounts, an incomplete singleton, changing Exit A accounts and a coordinator correction.

The main workflow is to start a fresh replay, advance the reports, select an incident, inspect source text and evidence, then confirm or correct its membership. The Evaluation page shows the measured method, grouping metrics, explicit checks and limitations. Earlier event history remains available when another exercise begins.

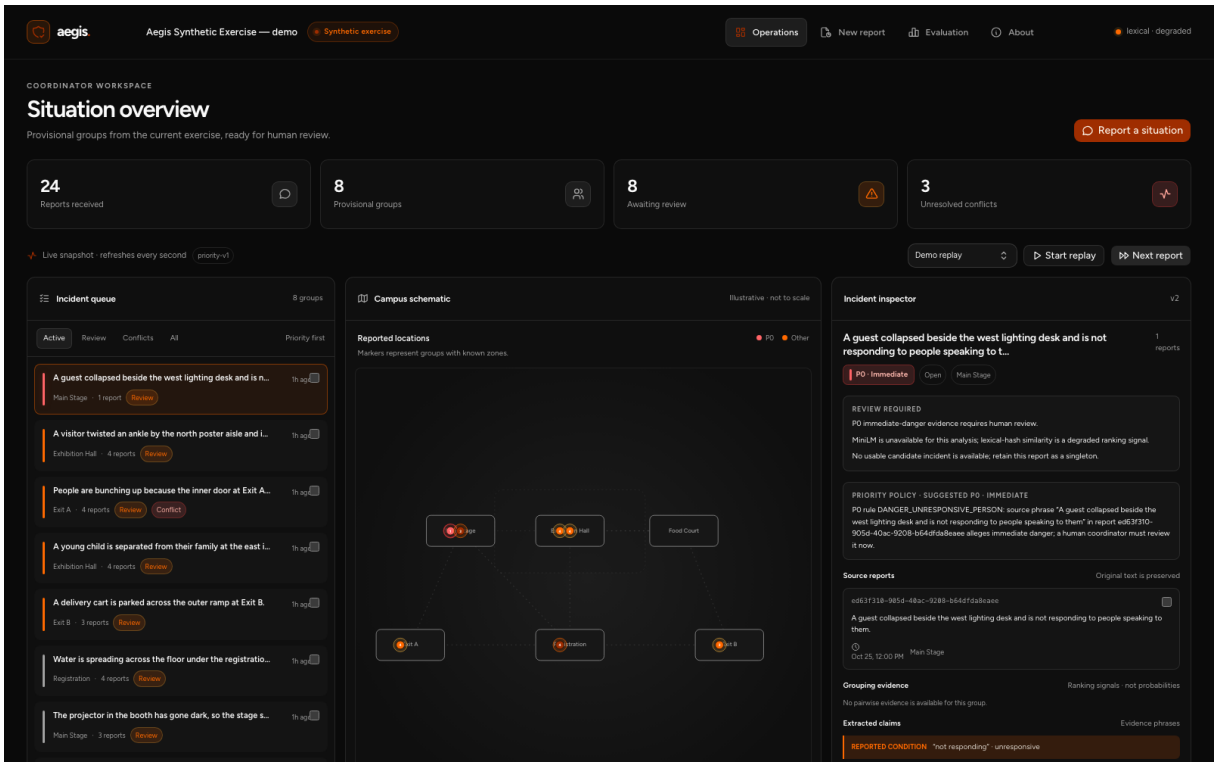


Figure 2: Operations after the 24-report demo replay. The board shows eight provisional groups and the active lexical matching mode.

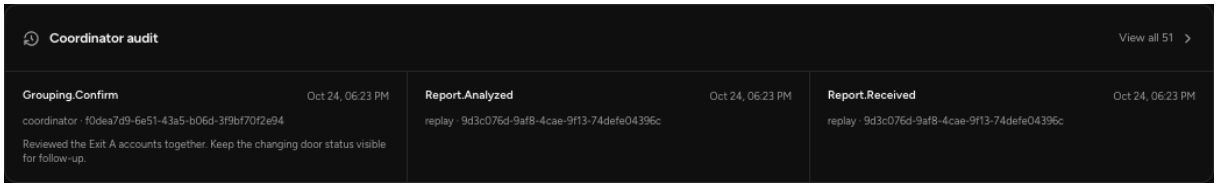


Figure 3: Audit entries after a coordinator confirms selected membership and records a reason.

6 Evaluation

6.1 Dataset and protocol

The versioned fixtures contain 144 reports across three sets, as shown in Table 4. Runtime inputs and ground-truth labels are stored separately. The evaluator reads the fixed test inputs and labels, and the matcher receives only report content and context.

Table 4: Synthetic fixtures used by the project

Set	Reports	Families	Purpose
Demo	24	8	Step-controlled demonstration
Development	60	20	Threshold and configuration development
Test/regression	60	20	Fixed grouping and policy measurements

Categories and scenario patterns recur across sets. Test cases were inspected during engineering, and discovered failures became regression cases. These measurements describe a small synthetic

regression benchmark. They cannot establish accuracy across independent events, languages or operational conditions.

6.2 Metrics and baseline

Grouping is evaluated over unordered report pairs. A true positive (TP) is a same-family pair placed together. A false positive (FP) is a different-family pair placed together, counted as a false merge pair. A false negative (FN) is a same-family pair left apart, counted as a missed pair. The metrics are

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}. \tag{2}$$

The review rate is the fraction of reports carrying a review reason. Its interpretation depends on the method's flagging rules.

The baseline uses token-set Jaccard similarity with simple zone and time gates. It is compared with Aegis contextual lexical grouping on the same inputs. The baseline's limited review flags do not establish that the remaining reports were safe to accept.

6.3 Measured grouping results

Table 5 reproduces the saved executable measurement in `evaluation/results/latest.json`, generated on 24 October 2026. No semantic encoder result is available for this environment.

Table 5: Pairwise results on the 60-report test/regression set

Measure	Jaccard baseline	Aegis lexical
Precision	1.000	1.000
Recall	0.061	0.621
F1	0.114	0.766
False merge pairs	0	0
Missed pairs	62	25
Reports flagged for review	8.3%	100%

The label set contains 66 same-family pairs. The baseline recovers four and Aegis recovers 41. Aegis therefore finds 37 additional same-family pairs while maintaining zero observed false merge pairs on this fixture. Its 25 missed pairs and 100% review rate show the remaining grouping and review workload. Perfect observed precision on this small set is limited evidence about future inputs.

The saved run measured median analysis-plus-incremental-grouping latency of 11.843 ms and a 95th percentile of 22.564 ms. These are process-local evaluator timings for this hardware and workload. They exclude an end-to-end browser response-time study.

6.4 Policy and retention checks

All six explicit evaluator checks passed: retention of all 60 reports, expected fixture cardinality, the explicit P0 case, the negative priority control, visibility of the changing-report case, and reviewability of the incomplete case. The last four checks each contain one labelled case or group, which limits the breadth of this evidence.

The separate API replay retained 24 reports in eight provisional groups, exposed three opposing report-pair conflicts and recorded 50 audit entries before an additional coordinator correction. The conflict count describes evidence pairs. It does not count independently verified events.

7 Implementation verification

The project verification record dated 24 October 2026 reports 196 passing backend tests and 140 passing frontend tests. Table 6 lists the recorded coverage totals. The backend scope includes the application and fixture generator. The frontend scope includes all source modules, component primitives and generated route registration.

Table 6: Recorded verification results

Measure	Executed / total	Coverage
Backend statements	2,414 / 2,414	100%
Backend branches	844 / 844	100%
Frontend statements	834 / 834	100%
Frontend branches	565 / 565	100%
Frontend functions	245 / 245	100%
Frontend lines	745 / 745	100%

Tests cover malformed payloads, exact source retention, idempotency conflicts, event isolation, analysis failure, grouping corrections, stale versions and concurrency races. Five integration tests execute the frontend's grouping payload builder with Bun and submit its output to FastAPI for confirm, reject, move, merge and split.

The recorded verification also includes Ruff lint and formatting, strict TypeScript checks, client and server production builds, and a Chromium smoke test against both services. The browser test advances all 24 replay reports, submits fresh text, confirms membership, inspects the audit and opens evaluation. Coverage measures execution of the included code paths. It leaves correctness for untested input distributions and operational conditions unresolved.

The API validates the Host header, checks browser origins for mutations, requires JSON POST bodies and returns non-caching responses. Production authentication remains future work. SQLite transactions and version checks address consistency within the prototype's local exercise model.

8 Limitations and further work

8.1 Current limits

The measured matcher depends on lexical overlap and a small set of English extraction rules. Paraphrases, unsupported negations, ambiguous references and unseen descriptions can affect grouping or priority. Optional semantic inference is implemented but remains unmeasured with actual MiniLM weights in the recorded environment.

The synthetic fixtures are small and share recurring categories and scenario structures. No independent deployment study measured coordinator workload, response time, injuries or other safety outcomes. The map is illustrative. The application assumes a single coordinator and local use, with no authenticated user roles or production operating policy.

8.2 Further work

The next evaluation should run the cached MiniLM model and lexical fallback on a larger independently prepared set. Useful additions include varied paraphrases, multilingual inputs, explicit temporal changes, challenging negative controls and error analysis by incident type. Thresholds should be fixed before the independent evaluation.

Interface evaluation should observe coordinators completing realistic exercises and measure correction effort, missed information and decision time. Access control, operational security review, backups, deployment monitoring and accessibility testing on real devices are required before broader use. Any operational priority policy would need approval from the responsible stakeholders.

The intended sustainability link is SDG 11 through coordination during community events, with secondary relevance to SDG 3. These are intended areas of relevance. The project has measured software behaviour and synthetic grouping outcomes.

9 Conclusion

Aegis implements a complete local workflow for receiving synthetic reports, proposing incident groups, inspecting evidence and recording coordinator corrections. Its persistence model keeps source accounts accessible through uncertainty, conflicts and analysis failure. The contextual lexical method recovers more same-family pairs than the Jaccard baseline on the fixed test set, while retaining every report for review. The current evidence supports the prototype's inspectable coordination workflow and defines the evaluation needed for further development.

References

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP-IJCNLP, 2019. arXiv:1908.10084.

[2] Sentence Transformers, "all-MiniLM-L6-v2." Hugging Face model card. Accessed 24 October 2026.

[3] Sentence Transformers, "Semantic Textual Similarity." Sentence Transformers documentation. Accessed 24 October 2026.

[4] Aegis project repository, implementation and verification materials, 24 October 2026. `backend/aegis/`, `frontend/src/`, `docs/API_CONTRACT.md`, `docs/VERIFICATION.md`, `evaluation/results/latest.json` and `evaluation/results/verification.json`.